ESAF Playwright Automation

Playwright suite for ESAF CRM and CBS, with a built-in browser-based **load testing harness** that produces an industry-standard performance report (percentiles, throughput, error rate, Apdex, SLO verdict, time series).

- Functional specs: `tests/CRM`, `tests/CBS`, `tests/AOCO`
- Load specs: `tests/load/crm`, `tests/load/cbs`, `tests/load/aoco`
- Full load-testing reference: `LOAD_TESTING.md`

---

1. Setup

```
npm install
npx playwright install chromium
```

Node 18+ is required (validated on Node 25).

---

2. Load test quick start

Each virtual user (VU) is a real Chromium session that logs in **once** and then loops its business flow until the duration or iteration cap is reached. Every step is timed as a named transaction.

Always start with a smoke run to validate credentials, data and selectors:

```
npm run load:smoke          # CRM: 2 VUs per flow, 2 iterations each
```

Then run the real profile:

```
npx cross-env LOAD_TEST=1 VUS=10 RAMP_UP_SEC=120 DURATION_SEC=600 npx playwright test tests
```

Open the report when it finishes:

```
open load-results/<runId>/index.html          # macOS
start load-results\\<runId>\\index.html         # Windows
```

> `LOAD_TEST=1` is what switches Playwright into load mode: workers scale to the VU count, retries/trace/video/screenshots are disabled so they cannot skew timings, and the aggregated report is generated in the global teardown. All `npm run load:*` scripts use `cross-env` so they work on Windows, macOS and Linux. When invoking `npx playwright test` directly, prefix with `npx cross-env` as shown below.

---

3. Available flows

Flow	Key	Transactions
CRM CASA 360	`casa360`	Login, search account, open details, Financial Details tab
CRM Customer 360	`customer360`	Login, search customer, open details, Communication tab, KYC Info tab
CRM Deposit 360	`deposit360`	Login, search deposit, open details, Financial Details tab, Payout Instructions tab
CRM Service Request	`srMobileUpdate`	Login, search customer, create SR (CIF / PII), enter new mobile number, save
AOCO CIF Alone	`aocoCifAlone`	Login, open new AO_Introduction (popup), select CIF Alone constitution
CBS Balance Enquiry	`balanceEnquiry`	Login, open task 7100, query customer balances, back to list
CBS Cash Deposit	`cashDeposit`	Login, open task 1401, enter transaction, denomination details, authorise
CBS Cash Withdrawal	`cashWithdrawal`	Login, open task 1001, enter withdrawal, denomination details, authorise
CBS CASA Transfer	`casaTransfer`	Login, open task 1091, debit/credit accounts, authorise
CBS NEFT Payment	`neftPayment`	Login, open task 2057, enter payment, operating instruction, service charges
CBS RTGS Payment	`rtgsPayment`	Login, open task 2055, enter payment, operating instruction, service charges
CBS IMPS Payment	`impsPayment`	Login, open task 2055 (P2A), enter payment, operating instruction, service charges
CBS Loan Installment	`loanPayment`	Login, open task 7022, installment enquiry, debit CASA (1065), authorise
CBS TD Quick Payin	`tdQuickPayin`	Login, open task 1021, payin by GL, missing signature, authorise

Flow	Key	Transactions
CBS TD Redemption	`tdRedemption`	Login, open task 7201, select deposit, credit CASA, missing signature, authorise
CBS Nominee Maintenance	`nomineeMaintenance`	Login, open task BA525, open account, add nominee
CBS Passbook Reissue	`passbookReissue`	Login, open task 7030, reissue passbook with a unique number

Specs are grouped per application: `tests/load/crm/`, `tests/load/cbs/`, `tests/load/aoco/`. The folder scopes collection; `SYSTEMS` / `FLOWS` decide what actually runs.

- `SYSTEMS=cbs` — every flow of that application (including its write flows).
- `FLOWS=cashDeposit,casa360` — individual flows.
- Both are additive; with neither set the default is `casa360,customer360`.
- **Read-only flows** — `casa360`, `customer360`, `deposit360`, `balanceEnquiry`. Use these for sustained, high-concurrency capacity tests.
- **Write flows** — everything else (`WRITE_FLOWS` in `config/loadProfile.js`): each iteration posts a real transaction, raises a service request or creates onboarding data.

---

4. Commands

```
# CRM — read-only flows
npm run load                        # all default flows, configured profile
npm run load:smoke                  # 2 VUs per flow, 2 iterations
npm run load:casa360
npm run load:customer360
npm run load:deposit360

# Whole application in one go (SYSTEMS + the matching folder)
npm run load:crm                    # every CRM flow, incl. the SR write flow
npm run load:cbs                    # every CBS flow
npm run load:aoco                   # every AOCO flow
npm run load:crm:smoke              # 1 VU, 1 iteration per flow of that app
npm run load:cbs:smoke
npm run load:aoco:smoke

# CRM Service Request — WRITES DATA, saves a real SR per iteration
npm run load:srmobileupdate

# AOCO onboarding — WRITES DATA, opens a new application per iteration
npm run load:aoco:cif:smoke         # 1 VU, 1 application — always run this first
npm run load:aoco:cif

# CBS Balance Enquiry — read-only
npm run load:balanceenquiry:smoke
npm run load:balanceenquiry

# CBS Cash Deposit — WRITES DATA, posts and authorises real deposits
npm run load:cashdeposit:smoke      # 1 VU, 1 deposit — always run this first
npm run load:cashdeposit            # configured profile

# Remaining CBS flows — ALL WRITE DATA
npm run load:cashwithdrawal
npm run load:casatransfer
npm run load:neft
npm run load:rtgs
npm run load:imps
npm run load:loanpayment
npm run load:tdquickpayin
npm run load:tdredemption
npm run load:nominee
npm run load:passbook


# Mixed CBS load: read-heavy enquiries plus a lighter posting load
npx cross-env LOAD_TEST=1 FLOWS=balanceEnquiry,cashDeposit \
  VUS_BALANCEENQUIRY=8 VUS_CASHDEPOSIT=2 \
  RAMP_UP_SEC=60 DURATION_SEC=600 npx playwright test tests

# Watch a run in a visible browser
npx cross-env LOAD_TEST=1 HEADLESS=false VUS=1 ITERATIONS=1 npx playwright test tests

# Rebuild the report without re-running the test
npm run load:report                           # latest run
npx cross-env LOAD_RUN_ID=20260818-143000 npm run load:report
```

```
# Functional (non-load) runs — load specs are excluded automatically
npm test
npm run test:headed
```

Fully specified example

```
npx cross-env LOAD_TEST=1 \
  FLOWS=casa360,customer360 \
  BASE_URL=https://dr-crm.esafbank.org \
  TEST_ENV=dr \
  VUS=25 RAMP_UP_SEC=300 DURATION_SEC=1800 \
  THINK_TIME_MIN_MS=1000 THINK_TIME_MAX_MS=3000 \
  SLO_P95_MS=5000 SLO_P99_MS=8000 SLO_ERROR_RATE_PCT=2 APDEX_T_MS=3000 \
  npx playwright test tests
```

Distributing a run across multiple VMs

When you need more VUs than one machine can drive, run the same script on multiple VMs. Two env vars split the test data so each VM logs in as a distinct set of users — no two VMs ever share a user:

Variable	Default	Meaning
`VM_ID`	`0`	0-based index of this VM
`VM_COUNT`	`1`	Total number of VMs running the same script

When `VM_COUNT > 1`, each VM takes a contiguous slice of the CSV's unique users: VM 0 gets users 0..VUS-1, VM 1 gets VUS..2*VUS-1, and so on. The CSV must contain at least `VM_COUNT * VUS` unique users. Metrics files are prefixed with `vm<vmId>-` so results from different VMs never collide, and every sample carries a `vmId` field for post-run aggregation.

Checkers are per-VM (each VM runs its own `CHECKER_VUS` checkers against its own local file queue), so checker VUs do not need to be sliced.

Example: 4 VMs, 25 maker VUs + 2 checker VUs each (100 makers + 8 checkers)

```
# VM 0
VM_ID=0 VM_COUNT=4 VUS=25 CHECKER_VUS=2 npm run load:cbs:composite:checker

# VM 1
VM_ID=1 VM_COUNT=4 VUS=25 CHECKER_VUS=2 npm run load:cbs:composite:checker

# VM 2
VM_ID=2 VM_COUNT=4 VUS=25 CHECKER_VUS=2 npm run load:cbs:composite:checker

# VM 3
VM_ID=3 VM_COUNT=4 VUS=25 CHECKER_VUS=2 npm run load:cbs:composite:checker
```

Example: 4 VMs, 25 CRM VUs each (100 makers total)

```
# VM 0
VM_ID=0 VM_COUNT=4 VUS=25 npm run load:crm:composite

# VM 1
VM_ID=1 VM_COUNT=4 VUS=25 npm run load:crm:composite

# VM 2
VM_ID=2 VM_COUNT=4 VUS=25 npm run load:crm:composite

# VM 3
VM_ID=3 VM_COUNT=4 VUS=25 npm run load:crm:composite
```

Merging results from multiple VMs

Each VM produces its own `load-results/<runId>/` directory. The merge script combines all raw samples into a single unified report with the same format as a single-VM run:

```
# Copy each VM's run directory into load-results/ on the aggregation machine,
# then merge them:
npm run load:merge -- load-results/20260831-100000 load-results/20260831-100001

# Custom merged run ID:
```

```
npx cross-env LOAD_RUN_ID=merged-20260831 npm run load:merge -- \
  load-results/vm1-20260831-100000 load-results/vm2-20260831-100001
```

The merged report lands in `load-results/<mergedRunId>/` and contains the same artefacts as a single-VM run (`index.html`, `summary.json`, CSVs, screenshots). VU numbers are offset per source VM so each VU remains distinct in the aggregate.

---

5. Test data

Data lives in CSVs. Rows are **grouped by `username`/`password`**: a VU is assigned one user, logs in once as that user, and cycles only through that user's records — so the session and the data always belong to the same user.

Flow	File	Columns
`casa360`	`fixtures/casa360.csv`	`username,password,casaNumber`
`customer360`	`fixtures/customer360.csv`	`username,password,customerId`
`deposit360`	`fixtures/crm_deposit360.csv`	`username,password,depositNumber`
`srMobileUpdate`	`fixtures/crm_srMobileUpdate.csv`	`username,password,customerId,newMobileNumber,srType,category,subCat`
`aocoCifAlone`	`fixtures/aoco_cifAlone.csv`	`username,password,constitution`
`balanceEnquiry`	`fixtures/cbs_balanceEnquiry.csv`	`username,password,customerId`
`cashDeposit`	`fixtures/cbs_cashDeposit.csv`	`username,password,accountNo,amount,narrative,noteDenomination,autho`
`cashWithdrawal`	`fixtures/cbs_cashWithdrawal.csv`	`username,password,accountNo,amount,noteDenomination,authorizer`
`casaTransfer`	`fixtures/cbs_casaTransfer.csv`	`username,password,fromAccountNo,amount,toAccountNo,authorizer`
`neftPayment`	`fixtures/cbs_neftPayment.csv`	`username,password,accountNo,amount,beneAccountNo,beneIfsc`
`rtgsPayment`	`fixtures/cbs_rtgsPayment.csv`	`username,password,casaAccountNo,amount,beneIfsc,beneAccountNo`
`impsPayment`	`fixtures/cbs_impsPayment.csv`	`username,password,accountNo,amount`
`loanPayment`	`fixtures/cbs_loanPayment.csv`	`username,password,loanAccountNo,casaAccountNo,amount,authorizer`
`tdQuickPayin`	`fixtures/cbs_tdQuickPayin.csv`	`username,password,accountNo,glAccountNo,amount,termMonths,termDays,`
`tdRedemption`	`fixtures/cbs_tdRedemption.csv`	`username,password,tdAccountNo,casaAccountNo,authorizer`
`nomineeMaintenance`	`fixtures/cbs_nomineeMaintenance.csv`	`username,password,accountNo,nomineeName,dateOfBirth,sharePercentage`
`passbookReissue`	`fixtures/cbs_passbookReissue.csv`	`username,password,accountNo,newPassbookNo`

Lines starting with `#` are comments. Override any path with `<FLOW>_DATA`, e.g. `CASA360_DATA`, `CASHDEPOSIT_DATA`, `TDREDEMPTION_DATA`.

Before any sizeable run, add one application user per VU. VUs are assigned users round-robin, so 25 VUs against 2 users puts ~13 concurrent sessions on each user; you would be measuring server-side session/lock contention rather than capacity. This is critical for `cashDeposit`, where CBS teller tills and maker/checker queues are stateful per user.

For `cashDeposit`, `amount` must be exactly divisible by `noteDenomination` — the note count is derived as `amount / noteDenomination` and validated before the run starts.

Data that can only be consumed once needs enough rows for the planned iterations:

- **`tdRedemption`** — a full redemption closes the deposit, so seed one TD per iteration.
- **`nomineeMaintenance`** — accounts accept a limited number of nominees.
- **`passbookReissue`** — the spec appends VU and iteration to `newPassbookNo` to keep it unique, so the CSV value is a prefix rather than the final number.

For `srMobileUpdate`, `filePath` is the SR attachment and is resolved relative to the repository root (`JMeterDocument.docx` by default).

---

6. Key environment variables

Variable	Default	Meaning
`LOAD_TEST`	–	Must be `1` to enable load mode
`SYSTEMS`	–	Applications to run in full, comma separated: `crm`, `cbs`, `aoco`
`FLOWS`	`casa360,customer360`	Flows to run, comma separated — any of the keys in section 3; additive with `SYSTEMS`
`VUS`	`10`	Virtual users per flow
`VUS_<FLOW>`	`VUS`	Per-flow VU override, e.g. `VUS_CASHDEPOSIT=2`

Variable	Default	Meaning
`VM_ID`	`0`	0-based index of this VM when splitting data across VMs
`VM_COUNT`	`1`	Total number of VMs running the same script (each takes a unique slice of users)
`<FLOW>_TASK_CODE` / `<FLOW>_TASK_LABEL`	per flow	Override the CBS task a flow opens
`RAMP_UP_SEC`	`60`	VUs start linearly across this window
`DURATION_SEC`	`600`	Steady-state duration per VU
`ITERATIONS`	`0` (duration driven)	Hard cap on iterations per VU
`THINK_TIME_MIN_MS` / `THINK_TIME_MAX_MS`	`1000` / `3000`	Pause between iterations
`BASE_URL`	`https://dr-crm.esafbank.org`	CRM under test
`CBS_BASE_URL`	`https://dr-cbs-uat-wll.esafbank.org:31108/obxlogin/login.html`	CBS under test
`AOCO_BASE_URL`	`https://dr-onboardingapp.esafbank.org/omniapp/pages/login/loginapp.app`	AOCO onboarding under test
`TEST_ENV`	`dr`	Environment label stamped on the report
`HEADLESS`	`true`	`false` to watch the browsers
`SLO_P95_MS` / `SLO_P99_MS` / `SLO_ERROR_RATE_PCT`	`5000` / `8000` / `2`	Report pass/fail thresholds
`APDEX_T_MS`	`3000`	Apdex satisfied threshold
`LOAD_RUN_ID`	timestamp	Run id / output folder name

Full list in `LOAD_TESTING.md`.

---

7. Results

Everything is written to `load-results/<runId>/`:

Artefact	Use
`index.html`	**Start here** — KPIs, SLO verdict, charts, all tables
`summary.json`	Machine-readable report for CI gates and trending
`transactions.csv`	Per transaction: count, pass/fail, min/avg/median/p90/p95/p99/max, std dev, TPS, Apdex
`flows.csv`	Same statistics per flow
`timeseries.csv`	Active VUs, throughput, errors and p95 over time
`samples.csv`	Raw log, one row per transaction execution
`errors.csv`	Grouped failures with counts and first/last seen
`http-metrics.csv`	Requests, 4xx, 5xx, failed requests, MB received, slowest request
`browser-metrics.csv`	TTFB, DOM interactive, DCL, load, FCP (avg and p95)
`screenshots/`	One screenshot per failed iteration

How to read it

1. **Error rate first** — high error rates invalidate the response times.
2. **p95 / p99** — the headline numbers; averages hide the tail.
3. **Throughput vs active VUs** — when throughput flattens while VUs keep rising, the system has saturated. That knee is the capacity limit.
4. **Re-login count** — non-zero means the server dropped sessions under load.
5. **HTTP 5xx / failed requests** — server-side saturation signals.

Client-side numbers alone are not a diagnosis: capture application server, DB and infrastructure metrics over the same window and attach them to the run folder.

---

8. Capacity and safety notes

- **One VU = one Chromium instance** (~250–400 MB RAM, ~0.3 vCPU). Beyond roughly 20–25 VUs on a laptop the client is the bottleneck and you are measuring your own machine. Distribute across agents for higher concurrency.

- `cashDeposit` **writes data.** Every iteration posts and authorises a real deposit; a 5 VU / 10 minute run creates hundreds of postings and moves till and account balances. Run only against UAT/DR with sign-off and an agreed clean-up plan. `balanceEnquiry` is read-only and carries no such risk.
- **CBS runs in a popup.** Login opens the Oracle Flexcube Retail window; the harness switches all transactions and metrics onto it (`utils/cbsSession.js`).
- `load-results/` is git-ignored. Archive the folder if you need to keep a run.

---

9. Troubleshooting

Symptom	Cause / fix
`FLOWS must contain at least one of: ...`	Typo in `FLOWS`; keys are case-sensitive (`cashDeposit`)
`SYSTEMS contains unknown value(s) ...`	Valid values are `crm`, `cbs`, `aoco`
Every test reports `disabled via FLOWS`	You scoped the folder but not the selection — add `SYSTEMS=<app>` or `FLOWS=<flow>`
`... must contain username and password columns for every row`	A data CSV row is missing credentials
`amount ... is not divisible by noteDenomination ...`	Fix `amount`/`noteDenomination` in `fixtures/cbs_cashDeposit.csv`
`[load-report] No samples found`	Run ended before any transaction completed, or wrong `LOAD_RUN_ID`
Every VU fails at `00 Login`	Wrong credentials, wrong `BASE_URL`/`CBS_BASE_URL`, or the environment is down
Timings look implausibly slow	Client saturation — reduce `VUS` or check machine CPU/RAM during the run